

Gym Membership Management System

Two design patterns made visible in a working gym back-office

Course: SEN3006 — Software Architecture

Presenter: Elif

Date: June 2026

The Problem Domain

A gym back-office has to keep track of **members**, **fitness classes**, and **who is enrolled in what**.

Two recurring engineering problems show up immediately:

- **Rich construction.** A fitness class has a name, an instructor, a day, a start time, a duration, a capacity, a room, a difficulty, an equipment list, a description. Ten fields. Most are optional. How do you construct that without a 10-argument constructor?
- **Change fan-out.** When a member enrolls in a class, several listeners want to know — the console, an audit log, an in-memory journal. How does the gym tell them all without hard-coding each one?

The patterns we chose — **Builder** and **Observer** — were not picked off a list. The domain forced them.

What the System Does

Five operations, exposed by the `Gym` class:

1. **Add member** — name and email; the gym assigns the ID.
2. **Remove member** — by ID; drops them from every class first.
3. **Add class** — built via `FitnessClass.Builder` ; duplicate names rejected.
4. **Remove class** — by name, case-insensitive.
5. **Enrol / drop** — capacity-checked, duplicate-checked.

Three **observers** watch everything:

- `ConsoleObserver` — prints each event to the screen.
- `AuditFileObserver` — appends each event to `audit.log` .
- `InMemoryJournalObserver` — keeps a queryable list in RAM.

No GUI. No fake notifications. No fake users. **Just the patterns, working.**

Architecture in One Diagram

The 16 source files fall into **three logical layers**:

Domain layer

`Member` · `FitnessClass` · `FitnessClass.Builder` · `Difficulty`

Pattern layer

`GymEvent` (*abstract*) and 6 concrete events:

`MemberAddedEvent` , `MemberRemovedEvent` , `ClassAddedEvent` , `ClassRemovedEvent` ,
`MemberEnrolledInClassEvent` , `MemberDroppedFromClassEvent`

`GymEventObserver` (*interface*) and 3 implementations:

`ConsoleObserver` , `AuditFileObserver` , `InMemoryJournalObserver`

Subject layer

`Gym` — owns members, classes, observers; **the only class that publishes events.**

`Main.java` is the scripted demo. Zero external dependencies. Default Java package.

Why Builder?

Builder is a creational pattern that separates the *construction* of a complex object from its *representation*, so the same construction process can yield different results step by step.

What it solves here. `FitnessClass` has 10 fields. 8 are optional. The call site stays readable:

```
FitnessClass yoga = new FitnessClass.Builder("Yoga Flow", "Sarah Lin")
    .dayOfWeek(DayOfWeek.MONDAY)
    .startTime(LocalTime.of(18, 30))
    .capacity(20)
    .room("Studio A")
    .difficulty(Difficulty.BEGINNER)
    .addEquipment("Yoga mat")
    .build();
```

What we rejected:

- *10-argument constructor*. Unreadable at the call site — you cannot tell which `int` is capacity and which is duration.
- *Telescoping constructors*. Combinatorial explosion for every optional field.
- *Setter-based POJO*. Breaks immutability — once a class is on the schedule, its capacity should not

Why Observer?

Observer is a behavioral pattern that defines a one-to-many dependency: when the subject changes, all registered observers are notified automatically.

What it solves here. Three log targets all want to know when a member enrolls, a class is added, anything. Without Observer, `Gym` would call each one directly — adding a fourth means editing `Gym`, breaking the Open/Closed principle.

```
// inside Gym – the ONLY place onEvent is called in the whole codebase
private void publish(GymEvent event) {
    for (GymEventObserver o : observers) {
        o.onEvent(event);
    }
}
```

What we rejected:

- *Direct method calls.* Tight coupling; cannot add a fourth observer without editing `Gym`.
- *Mediator pattern.* Centralises *control* — we want centralised *publication*. Different problem.
- *External pub/sub framework (Kafka, RxJava).* Extra dependency. Massive overkill for our scale.

SOLID — Quick Pass

- **S — Single Responsibility.** `Gym` coordinates. `FitnessClass.Builder` constructs. Each observer consumes. No class wears two hats.
- **O — Open/Closed.** Add a new event = one new class. Add a new observer = one new class. `Gym` is *not edited* either time.
- **L — Liskov Substitution.** Every `GymEvent` subclass is substitutable wherever `GymEvent` is expected. Every observer is substitutable wherever `GymEventObserver` is expected.
- **I — Interface Segregation.** `GymEventObserver` has one method, `onEvent(GymEvent)`. No observer is forced to implement anything it does not use.
- **D — Dependency Inversion.** `Gym` depends on the `GymEventObserver` *interface* and the abstract `GymEvent` — never on a concrete observer or concrete event subclass.

Live Demo — copy these commands

```
cd path\to\gym-management-system  
run.bat  
type audit.log
```

What to watch for as the output scrolls:

- Section 2 — three classes built via `FitnessClass.Builder`.
- Section 4 — each enrolment fires **one** event, three observers see it.
- Section 6 — three intentional errors, each caught with a plain-English message.
- Section 7 — the journal dumps every event captured in memory.
- The last line — `audit.log` written with 12 lines.

Extension Story

The Open/Closed payoff, measured in **lines of code**:

Add a Slack notifier?

- One new file: `SlackObserver.java` implementing `GymEventObserver` .
- One new line in `Main` : `gym.addObserver(new SlackObserver());`
- Edits to `Gym.java` : **zero**.

Add a "class rescheduled" event?

- One new file: `ClassRescheduledEvent.java` extending `GymEvent` .
- One new method on `Gym` that calls `publish(new ClassRescheduledEvent(...))` .
- Edits to existing observers: **zero** — they handle any `GymEvent` by contract.

That is what Open/Closed buys us. *New behavior is additive, not invasive.*

Recap

- We modelled a small, honest gym domain — members, classes, enrolments.
- **Builder** handled the rich-construction problem — `FitnessClass` has 10 fields, 8 optional.
- **Observer** handled the fan-out problem — three observers learn about every event through one `publish` call.
- SOLID held: *adding a new event or a new observer never touches* `Gym`.
- Both patterns were chosen because the domain forced them — **not because the rubric required them.**

Q&A — Likely Questions

Q: Why not Lombok's `@Builder` annotation?

Zero external dependencies. The brief asks for hand-written Java patterns.

Q: Why is `FitnessClass` immutable?

A class on the schedule should not silently mutate after publication. Capacity, room, and time are facts, not variables.

Q: Why an *interface* for `GymEventObserver`, not an abstract class?

There is no shared state between observers — only shared behavior. The interface communicates that.

Q: Why typed event classes instead of `notify(String message)` ?

Keeps presentation out of the domain. Each observer formats events in its own voice — the console with brackets, the audit file with newlines, the journal with no formatting at all.

Thank you.